

→ Inheritance :-

~~Birth class~~

Inheritance is the mechanism by which one class acquires the **properties** and **methods** of another class.

It is mainly used for **code reusability** and establishing a relationship between classes.

→ Basic Concept :

Suppose we ~~are~~ have a parent class :

```
Class Animal {
```

```
    void eat {
```

```
        sout("Animal eats");
```

```
    }
```

```
}
```

child class :

```
class Dog extends Animal {
```

```
    void bark() {
```

```
        sout("Dog Barks");
```

```
    }
```

```
}
```

Now :

```
Dog d = new Dog();
```

```
d.eat();
```

```
d.bark();
```

output :

Animal eats

Dog barks

Although `eat()` is defined in `Animal`, the `Dog` object can use it because `Dog` inherits from `Animal`.

→ Parent class :

The class whose properties are inherited.

Also called :

- Super Class
- Base class
- Parent class

→ Child class :

The class that inherits from the parent.

Also called :

- Subclass
- Derived class
- Child class

→ Syntax of Inheritance:

Java uses the **extends** keyword.

```
class child extends parent {  
}
```

full eg:~

```
class Animal {  
    void eat() {  
        sout("Animal eats");  
    }  
}
```

```
class dog extends Animals {  
    void bark() {  
        sout("Dog barks");  
    }  
}
```

```
public class pogo {  
    psun {  
        Dog d = new Dog();  
        d.eat();  
        d.bark();  
    }  
}
```

→ Why do we need inheritance?
The biggest advantage is **code reusability**.

Without inheritance:

```
class dog {  
    void eat() {  
        cout << "Animal eats";  
    }  
  
    void bark() {  
        cout << "Dog barks";  
    }  
}
```

```
class dog extends Animal {  
    void bark() {  
        cout << "Dog bark";  
    }  
}
```

If we have cat, we may have to write eat() again.

With inheritance:

```
class Animal {  
    void eat() {  
        cout << "Animal eats";  
    }  
}
```

class Dog extends Animal {

void bark() {

 System.out.println("Dog barks");

}

}

class Cat extends Animals {

void meow() {

 System.out.println("Cat meows");

}

}

Now :

Animal

/ \

Dog

Cat

Both Dog and Cat automatically get:

eat()

→ Types of inheritance in java
 There are mainly five commonly discussed structures:

1. Single inheritance
2. Multilevel inheritance
3. Hierarchical inheritance
4. Multiple inheritance
5. Hybrid inheritance

however, java does not support multiple inheritance through classes.

(i) Single inheritance:

One child class inheritance from one parent class.

A

↓

B

Eg:

```
Class Animal {
    void eat() {
        sout("Animals eat");
    }
}
```

```
Class Dog extends Animal {
    void bark() {
        sout("barking");
    }
}
```

here;

Animal → Parent

Dog → child

(ii) Multilevel Inheritance:

A class inherits from another child class, forming a chain.

A



B



C

```
eg; Class Animal {  
    void eat() {  
        cout << "Eating";  
    }  
}
```

```
Class Dog extends Animal {  
    void bark() {  
        cout << "barking";  
    }  
}
```

```
Class cat extends Dog {  
    void meow() {  
        cout << "meow";  
    }  
}
```

Now :

```
cat c = new cat();
```

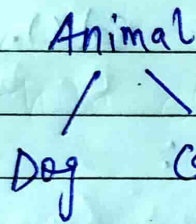
```
c.meow();
```

```
c.bark();
```

```
c.eat();
```

(ii) Hierarchical inheritance:

Multiple child classes inherit from the same parent class.



Eg;

```

class Animal {
    void eat() {
        cout << "Eating";
    }
}
  
```

```
}
```

```

class Dog extends Animal {
    void bark() {
        cout << "barking";
    }
}
  
```

```
}
```

```

class Cat extends Animal {
    void meow() {
        cout << "meow";
    }
}
  
```

```
}
```

```
}
```

Now :

```
Dog d = new Dog();  
Cat c = new Cat();
```

```
c.eat();
```

```
c.meow();
```

```
d.eat();
```

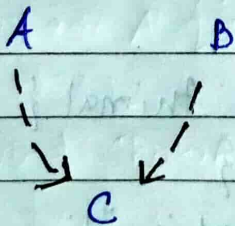
```
d.bark();
```

Both children inherit eat().

(iv) Multiple Inheritance :

Multiple Inheritance means one child class inherits from **more than one parent class**.

for example :



In java, this is **not allowed** using class.

This is invalid:

```
class A {  
}  
class B {  

```

```
class C extends A, B { //error  
}
```

→ Why doesn't java support it?

One major problem is **ambiguity**.

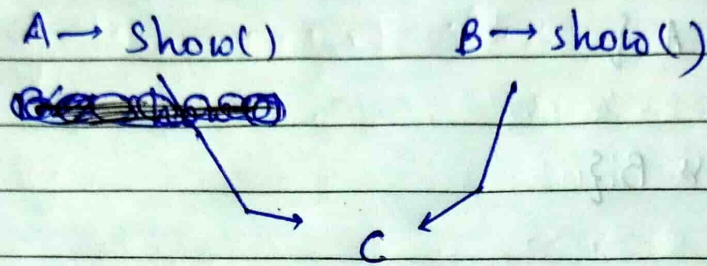
Suppose:

```
class A {  
    void show () {  
        sout("A");  
    }  

```

```
class B {  
    void show () {  
        sout("B");  
    }  
}
```

If C inherits both:



Then,

```
C c = new C();
```

which **show()** should execute?

A.show()?

B.show()?

This creates **Ambiguity**.

→ Multiple Inheritance using Interfaces:-
Java allows a class to implement multiple interfaces.

```
interface A {
```

```
    void showA();
```

```
}
```

```
interface B {
```

```
    void showB();
```

```
}
```

```
class C implements A, B {
```

```
    public void showA() {
```

```
        System.out.println("A");
```

```
    }
```

```
public void showB() {
    sout("B");
}
```

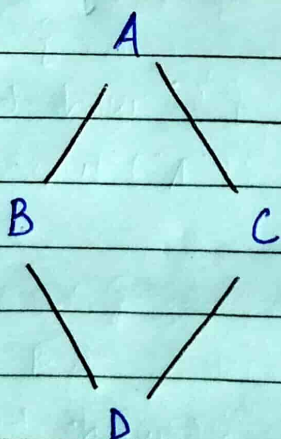
This is valid.

Class c implements A, B.

Java does not support multiple inheritance through class, but it supports multiple inheritance of type through interfaces.

(v) Hybrid inheritance :-

Hybrid inheritance is a combination of two or more types of inheritance.



Java does not support this structure directly through classes if it requires multiple inheritance.

It can be achieved using ~~interfacing~~ interface.



What is inherited?

A child class can inherit accessible member of its parent class.

Eg;

```
class Animal {
```

```
    int age;
```

```
    void eat() {  
        cout << "Eating";  
    }
```

```
}
```

```
class Dog extends Animal {
```

```
    void bark() {  
        cout << "Barking";  
    }
```

```
}
```

```
}
```

Dog can use:

d.age

d.eat()

d.bark()



→

What is NOT inherited?

Some members are not inherited in the usual sense.

① Private members:

```
class Animal {
```

```
    private int age;
```

```
class Dog extends Animal {
```

Dog cannot directly access:

d.age; // not possible

The private member belongs to Animal and is accessible only inside Animal.

② Constructor:

Constructors are not inherited.

```
class Animal {
```

```
    Animal() {
```

```
        cout << "Animal";
```

```
    }
```

```
}
```

Class Dog extends Animal {

3

Dog does not inherit Animal() constructor.

→ Super keyword in inheritance:

The super keyword refers to the **immediate parent class**.

It is used mainly to:

1. Access parent class variable
2. Call parent class method
3. ~~Call~~ Call parent class constructor

① Access parent variable :-

```

class Animal {
    int age = 10;
}

class Dog extends Animal {
    int age = 20;
    void display() {
        cout << age;
        cout << super.age;
    }
}
    
```

here, age → means current class variable

super.age → means super class variable

② Calling parent method:

```
class Animal {
```

```
    void sound {
```

```
        cout << "Animal sound";
```

```
    }
```

```
}
```

```
class Dog extends Animal {
```

```
    void sound() {
```

```
        super.sound();
```

```
        cout << "Dog barks";
```

```
    }
```

```
}
```

```
Dog d = new Dog();
```

```
d.sound();
```

Output:-

Animal sound

Dog Barks

super.sound calls the parent version of the method.

③ Calling parent Constructor:

Use:

Super();

Eg:

```
Class Animal {
```

```
    Animal() {
```

```
        cout << "Animal constructor";
```

```
    }
```

```
}
```

```
Class Dog Extends Animal {
```

```
    Dog() {
```

```
        Super();
```

```
        cout << "Dog constructor";
```

```
    }
```

```
}
```

```
Dog d = new Dog();
```

Output:

Animal constructor

Dog constructor